

Plateforme de D mat rialisation des Appels   Projets
et Gestion des Subventions

FDCUIC — Fonds de D veloppement des Cultures Urbaines
et des Industries Cr atives

Auteure	Fatou Biaye
Profil	D�veloppeur Web & Mobile
Niveau	BTS / DUT — D�veloppement Informatique
Ann�e	2024 – 2025
Statut	M�moire de fin d'�tudes

TABLE DES MATIÈRES

Remerciements	3
Résumé / Abstract	3
Introduction générale	4
1. Contexte et problématique	4
1.1 Présentation du FDCUIC	4
1.2 Problème identifié	4
1.3 Enjeux de la digitalisation	5
2. Objectifs du projet	5
3. Analyse des besoins	5
3.1 Acteurs du système	5
3.2 Besoins fonctionnels	6
3.3 Besoins non fonctionnels	6
4. Architecture du système	6
4.1 Architecture 3-tiers	6
4.2 Diagramme des composants	7
5. Choix technologiques	7
5.1 Backend : Node.js vs Laravel vs Django	7
5.2 Mobile : Flutter vs React Native	8
5.3 Base de données : PostgreSQL vs MongoDB	8
6. Modélisation des données	9
6.1 Entités principales	9
6.2 Schéma relationnel	9
7. API REST	10
7.1 Conception des endpoints	10
7.2 Format des échanges	10
8. Sécurité	11
9. Interface Utilisateur (UI/UX)	11
10. Déploiement	12
11. Défis techniques et solutions	12
12. Planning prévisionnel	13
Conclusion générale	13
Références bibliographiques	14

Remerciements

Je tiens à exprimer ma profonde gratitude à mon encadreur pédagogique pour ses conseils précieux et son soutien tout au long de l'élaboration de ce dossier. Mes remerciements vont également à l'équipe du FDCUIC pour les informations partagées sur leurs processus actuels, ainsi qu'à ma famille et mes camarades de promotion pour leur encouragement constant.

Résumé

Ce dossier technique présente la conception d'une plateforme numérique destinée à dématérialiser les processus d'appels à projets et de gestion des subventions du FDCUIC (Fonds de Développement des Cultures Urbaines et des Industries Créatives). La solution repose sur une architecture 3-tiers combinant un backend Node.js/Express, une base de données PostgreSQL, une application mobile Flutter et une interface web d'administration. L'objectif est de remplacer un processus entièrement manuel par un système centralisé, sécurisé et traçable.

Mots-clés : Dématérialisation, Gestion de subventions, Node.js, Flutter, PostgreSQL, API REST, JWT, Cultures urbaines, Industries créatives.

Introduction Générale

L'ère numérique transforme en profondeur les modes de gouvernance et de gestion administrative. Les institutions qui tardent à adopter les outils digitaux se retrouvent confrontées à des inefficacités croissantes : délais allongés, pertes d'information, manque de transparence et frustration des parties prenantes.

Le FDCUIC, en charge du financement des acteurs des cultures urbaines et des industries créatives au Sénégal, ne fait pas exception. Ses processus actuels d'appels à projets et d'attribution de subventions reposent intégralement sur des formulaires papier, des échanges par courrier et des tableaux de suivi manuels, générant des risques importants d'erreurs et un manque de visibilité pour les candidats comme pour les gestionnaires.

Ce dossier technique présente la conception complète d'une plateforme de dématérialisation répondant à ces problématiques. Il couvre l'analyse des besoins, les choix architecturaux et technologiques, la modélisation des données, la sécurisation du système ainsi que la stratégie de déploiement envisagée.

1. Contexte et Problématique

1.1 Présentation du FDCUIC

Le Fonds de Développement des Cultures Urbaines et des Industries Créatives (FDCUIC) est un organisme dont la mission est de soutenir financièrement les porteurs de projets actifs dans les domaines de la musique urbaine, du cinéma, du design, de la mode, du jeu vidéo, des arts numériques et des métiers de la création. Il publie périodiquement des appels à projets, instruit les dossiers reçus et attribue des subventions aux lauréats.

1.2 Problèmes identifiés

L'analyse du fonctionnement actuel du FDCUIC a révélé les dysfonctionnements suivants :

Problème	Impact	Fréquence
Dossiers papier perdus ou illisibles	Candidatures non traitées	Régulière
Absence de suivi en temps réel	Candidats sans information	Permanente
Double saisie manuelle des données	Erreurs humaines fréquentes	Systematique
Délais de traitement élevés (semaines)	Découragement des porteurs de projets	Chaque session
Aucune traçabilité des décisions	Litiges et contestations	Occasionnelle

1.3 Enjeux de la digitalisation

La transformation numérique du FDCUIC répond à trois enjeux majeurs : premièrement, un enjeu d'efficacité opérationnelle, en réduisant les délais de traitement et en automatisant les tâches répétitives. Deuxièmement, un enjeu de transparence et de confiance, en permettant aux candidats de suivre l'avancement de leur dossier en temps réel. Troisièmement, un enjeu de conservation et de sécurité des données, en remplaçant les archives papier par une base de données structurée et sauvegardée.

2. Objectifs du Projet

L'objectif général est de concevoir et développer une plateforme numérique intégrée permettant de gérer l'intégralité du cycle de vie d'un appel à projets FDCUIC.

#	Objectif spécifique	Indicateur de réussite
O1	Permettre la soumission en ligne des dossiers	Formulaire complet accessible 24h/24
O2	Automatiser l'accusé de réception	Email automatique sous 1 minute
O3	Centraliser le suivi des candidatures	Tableau de bord administrateur
O4	Sécuriser l'accès par rôle	3 profils : Candidat, Évaluateur, Admin
O5	Générer des rapports de synthèse	Export PDF/Excel des résultats
O6	Garantir la disponibilité du système	Uptime >= 99% en période d'appel

3. Analyse des Besoins

3.1 Acteurs du système

Trois types d'acteurs interagissent avec la plateforme :

Acteur	Rôle	Interface utilisée
Candidat	Soumet et suit ses projets	Application mobile (Flutter)
Évaluateur	Instruit et note les dossiers	Application web
Administrateur	Gère les appels, valide les décisions, génère les rapports	Application web

3.2 Besoins fonctionnels

- Authentification sécurisée et gestion des profils utilisateurs
- Publication et gestion des appels à projets (dates, critères, budget)
- Soumission de dossiers avec upload de pièces jointes (PDF, images)
- Suivi en temps réel du statut de chaque candidature
- Module d'évaluation avec grille de notation configurable
- Génération automatique de notifications (email / push)
- Tableau de bord statistiques pour les administrateurs
- Export des données en PDF et CSV

3.3 Besoins non fonctionnels

Critère	Exigence
Performance	Temps de réponse API < 500 ms pour 95% des requêtes
Sécurité	Chiffrement des données sensibles, authentification JWT
Disponibilité	Uptime >= 99% lors des sessions d'appel à projets
Maintenabilité	Code modulaire, documenté et versionné avec Git
Compatibilité mobile	Android 8+ (Flutter), iOS non prioritaire
Scalabilité	Architecture permettant d'ajouter des modules sans refonte

4. Architecture du Système

4.1 Architecture 3-tiers

La plateforme est organisée selon une architecture 3-tiers qui sépare clairement les responsabilités entre la couche de présentation, la couche métier et la couche de données.

Couche	Technologie	Rôle
Présentation (Frontend Web)	React.js	Interface administrateur et évaluateur
Présentation (Mobile)	Flutter / Dart	Application candidat (Android)
Métier (Backend)	Node.js + Express.js	Logique métier, API REST, validation
Données	PostgreSQL	Persistance et intégrité des données
Stockage fichiers	AWS S3 ou stockage local	Pièces jointes des dossiers
Notifications	Nodemailer + Firebase	Emails et notifications push

4.2 Description des flux de communication

Le candidat utilise l'application mobile Flutter pour soumettre son projet. La requête HTTP est transmise à l'API REST Node.js/Express qui valide les données, applique les règles métier et interagit avec la base PostgreSQL. Les pièces jointes sont stockées séparément. L'évaluateur et l'administrateur accèdent à la plateforme via un navigateur web (React.js). Toutes les communications sont sécurisées en HTTPS et authentifiées via des tokens JWT.

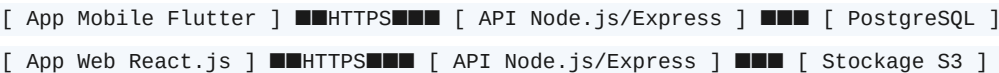


Figure 1 : Schéma simplifié de l'architecture

5. Choix Technologiques

5.1 Backend : Node.js vs Laravel vs Django

Critère	Node.js / Express	Laravel (PHP)	Django (Python)
Langage	JavaScript	PHP	Python
Performance E/S	Excellente (asynchrone)	Bonne	Bonne
Courbe d'apprentissage	Modérée	Faible	Modérée
Écosystème npm	Très riche	Riche (Composer)	Riche (pip)
Adapté API REST	Oui (natif)	Oui	Oui (DRF)

Choix retenu	✓ OUI	Non	Non
--------------	-------	-----	-----

Node.js a été retenu principalement pour sa gestion asynchrone non bloquante, idéale pour traiter de nombreuses requêtes simultanées lors des sessions d'appel à projets. De plus, l'utilisation de JavaScript côté serveur permet une cohérence avec les connaissances existantes de l'équipe de développement.

5.2 Mobile : Flutter vs React Native

Critère	Flutter (Dart)	React Native (JS)
Performance	Proche du natif (rendu Skia)	Bonne (bridge JS natif)
UI personnalisée	Totale (widgets propres)	Dépend des composants natifs
Taille de l'APK	Plus grande (~20 Mo)	Plus petite (~10 Mo)
Productivité	Hot reload rapide	Hot reload disponible
Choix retenu	✓ OUI	Non

Flutter a été sélectionné pour ses performances élevées et la cohérence visuelle qu'il offre entre les appareils. Son moteur de rendu propre (Skia) garantit une expérience utilisateur uniforme quel que soit le terminal Android utilisé par le candidat.

5.3 Base de données : PostgreSQL vs MongoDB

Critère	PostgreSQL	MongoDB
Type	Relationnelle (SQL)	Non-relationnelle (NoSQL)
Intégrité des données	Forte (contraintes FK, transactions)	Limitée
Requêtes complexes	Excellentes (JOIN, agrégats)	Limitées
Relations entre entités	Native et optimisée	Emulée (embedded docs)
Choix retenu	✓ OUI	Non

Le modèle de données du projet est fortement relationnel : un utilisateur peut avoir plusieurs projets, un projet est lié à un appel à projets, à des évaluations et à une subvention. PostgreSQL est le choix naturel pour garantir l'intégrité référentielle et permettre des requêtes d'analyse avancées.

6. Modélisation des Données

6.1 Entités principales

Entité	Attributs principaux	Description
User	id, nom, prenom, email, mot_de_passe_hash, role	Tout utilisateur de la plateforme
AppelAProjet	id, titre, description, date_ouverture, date_cloture, budget_maximal	Ses projets sont publiés par l'admin
Projet	id, titre, description, fichier_pdf, statut, date_soumission, appel_id	Dossier soumis par un candidat
Evaluation	id, note, commentaire, date, evaluateur_id, projet_id	Restriction d'un dossier par un évaluateur
Subvention	id, montant, date_attribution, statut_paiement, projet_id	Attribution financière à un projet lauréat
Notification	id, message, type, lu, date, user_id	Alerte envoyée à un utilisateur

6.2 Relations entre entités

Les relations entre entités respectent les formes normales (3NF) pour éviter la redondance des données :

- User (1) ■■■■ (N) Projet : un candidat peut soumettre plusieurs projets
- AppelAProjet (1) ■■■■ (N) Projet : un appel reçoit plusieurs projets
- Projet (1) ■■■■ (N) Evaluation : un projet est évalué par un ou plusieurs évaluateurs
- Projet (1) ■■■■ (1) Subvention : un projet lauréat reçoit une subvention
- User (1) ■■■■ (N) Notification : un utilisateur reçoit plusieurs notifications

7. API REST

7.1 Conception des endpoints

L'API suit les conventions REST : utilisation des verbes HTTP (GET, POST, PUT, DELETE), URLs en minuscules, versionnement via le préfixe `/api/v1/`, réponses en JSON.

Méthode	Endpoint	Description	Auth
POST	<code>/api/v1/auth/register</code>	Création de compte	Public
POST	<code>/api/v1/auth/login</code>	Connexion + token JWT	Public
GET	<code>/api/v1/appels</code>	Liste des appels à projets	Public
GET	<code>/api/v1/appels/:id</code>	Détail d'un appel	Public
POST	<code>/api/v1/projets</code>	Soumettre un projet	Candidat
GET	<code>/api/v1/projets/mes-projets</code>	Projets du candidat connecté	Candidat
GET	<code>/api/v1/projets/:id</code>	Détail d'un projet	Candidat/Admin
PUT	<code>/api/v1/projets/:id/statut</code>	Changer le statut d'un projet	Admin
POST	<code>/api/v1/evaluations</code>	Soumettre une évaluation	Évaluateur
GET	<code>/api/v1/admin/dashboard</code>	Statistiques globales	Admin
GET	<code>/api/v1/admin/export</code>	Export CSV des dossiers	Admin

7.2 Format des échanges JSON

Exemple de réponse de l'endpoint POST `/api/v1/auth/login` :

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 42,
    "nom": "Biaye",
    "prenom": "Fatou",
    "role": "candidat"
  }
}
```

Figure 2 : Exemple de réponse JSON du endpoint d'authentification

8. Sécurité

La sécurité est une priorité absolue, notamment pour protéger les données personnelles des candidats et l'intégrité des décisions d'attribution de subventions.

Menace	Mesure mise en place	Technologie
Vol de mot de passe	Hachage irréversible des mots de passe	bcrypt (salt 12)
Usurpation d'identité	Authentification par token à durée limitée	JWT (expire 24h)
Accès non autorisé	Contrôle d'accès basé sur les rôles (RBAC)	Roleware Express
Injection SQL	Requêtes paramétrées exclusivement	Sequelize ORM
Attaque XSS	Validation et nettoyage des entrées	express-validator
Écoute réseau	Chiffrement de toutes les communications	HTTPS / TLS 1.3
Fuite de fichiers	URLs d'accès temporaires et signées	Tokens d'accès S3

9. Interface Utilisateur (UI/UX)

La conception de l'interface est guidée par le principe de simplicité et d'accessibilité, en tenant compte du profil des utilisateurs finaux : artistes et porteurs de projets créatifs dont la maîtrise des outils numériques peut être variable.

Application mobile (Flutter) — Vue Candidat

Écran	Fonctionnalités
Accueil	Liste des appels à projets en cours, avec dates et budgets disponibles
Détail d'un appel	Description complète, critères, bouton 'Déposer un dossier'
Formulaire de dépôt	Saisie des informations du projet + upload de la pièce jointe PDF
Mes projets	Liste de tous les projets soumis avec leur statut coloré (En attente / Accepté / Rejeté)
Notifications	Historique des alertes reçues
Profil	Informations personnelles et modification du mot de passe

Application web (React.js) — Vue Admin/Évaluateur

Module	Fonctionnalités
Tableau de bord	KPIs : nombre de dossiers reçus, en cours, validés, rejetés
Gestion des appels	Créer, modifier, ouvrir/fermer un appel à projets
Dossiers reçus	Table filtrable et triable de toutes les candidatures
Instruction	Formulaire d'évaluation avec grille de notation et commentaires
Subventions	Suivi des attributions et des paiements
Rapports	Génération et export de rapports de synthèse

10. Déploiement

Composant	Solution envisagée	Justification
Backend Node.js	VPS Ubuntu (ex. DigitalOcean / Render)	Contrôle total, coût maîtrisé
Base de données	PostgreSQL managé (Supabase / Railway)	Mises à jour automatiques, haute disponibilité
Application web	Vercel / Netlify	Déploiement continu, CDN intégré
Application mobile	Distribution APK directe	Évite les délais du Play Store
Fichiers (PJ)	Stockage cloud (AWS S3 ou Cloudinary)	Scalable et sécurisé

Reverse proxy	Nginx	Gestion SSL, load balancing
---------------	-------	-----------------------------

La mise en production sera précédée d'une phase de tests en environnement de staging identique à la production, permettant de valider les performances et la sécurité avant le lancement officiel.

11. Défis Techniques et Solutions Envisagées

Défi	Description	Solution envisagée
Gestion des fichiers volumineux	Les dossiers PDF peuvent dépasser 10 Mo	Upload multipart + validation côté serveur, limite de 15 Mo
Pics de charge	Afflux massif lors de la clôture d'un appel	Mise en file d'attente (Bull.js) + cache Redis
Synchronisation offline	Zones à connectivité limitée au Sénégal	Mode hors-ligne Flutter + synchronisation différée
Sécurité des données	Données personnelles et financières sensibles	Chiffrement AES des données au repos, HTTPS en transit
Gestion des rôles	Droits différents selon le profil	Middleware RBAC avec vérification à chaque requête
Traçabilité des décisions	Audit des modifications administratives	Table de logs avec horodatage et identifiant de l'auteur

12. Planning Prévisionnel

Phase	Tâches principales	Durée estimée
Phase 1 — Analyse	Recueil des besoins, maquettes, modélisation BDD	2 semaines
Phase 2 — Backend	Mise en place API, authentification, CRUD complet	3 semaines
Phase 3 — Mobile	Développement Flutter : authentification, formulaires	3 semaines
Phase 4 — Web Admin	Interface React.js : dashboard, gestion dossiers	2 semaines
Phase 5 — Tests	Tests unitaires, intégration, tests utilisateurs	2 semaines
Phase 6 — Déploiement	Mise en production, documentation, livraison	1 semaine
TOTAL		~13 semaines

Conclusion Générale

Ce dossier technique a présenté de manière structurée la conception d'une plateforme numérique visant à moderniser la gestion des appels à projets et des subventions du FDCUIC. L'analyse des dysfonctionnements actuels a confirmé l'urgence et la pertinence de cette transformation digitale.

Les choix technologiques retenus — Node.js pour le backend, Flutter pour l'application mobile, PostgreSQL pour la base de données et React.js pour l'interface web — forment un stack cohérent, performant et adapté aux contraintes d'un projet de cette envergure. L'architecture 3-tiers garantit la maintenabilité et la scalabilité du système.

La mise en œuvre de cette plateforme permettra au FDCUIC de gagner significativement en efficacité, de renforcer la confiance des porteurs de projets grâce à la transparence du processus, et de disposer d'un outil de pilotage précis pour ses activités de financement des cultures urbaines et des industries créatives.

Perspectives : Dans une version ultérieure, la plateforme pourrait intégrer un module d'intelligence artificielle pour pré-analyser les dossiers soumis et détecter les candidatures incomplètes, ainsi qu'une API ouverte permettant aux partenaires institutionnels de se connecter directement au système.

Références Bibliographiques

- [1] Documentation officielle Node.js — <https://nodejs.org/en/docs>
- [2] Documentation Express.js — <https://expressjs.com>
- [3] Documentation Flutter — <https://docs.flutter.dev>
- [4] Documentation PostgreSQL 16 — <https://www.postgresql.org/docs>
- [5] JSON Web Tokens (JWT) — RFC 7519 — <https://jwt.io/introduction>
- [6] Principes REST — Roy Fielding, Architectural Styles and the Design of Network-based Software Architectures, 2000
- [7] OWASP Top Ten Security Risks — <https://owasp.org/www-project-top-ten>
- [8] Sequelize ORM Documentation — <https://sequelize.org/docs>
- [9] React.js Documentation — <https://react.dev>
- [10] Supabase (PostgreSQL managé) — <https://supabase.com/docs>